Sistema de telemetría y análisis estadístico de eventos para un videojuego Breakout

Daniel Manzanera, Manuel Hormechea, Ivan Ortega

Resumen— Se desarrolló un prototipo de telemetría para un videojuego tipo *Breakout*, con registro incremental de seis tipos de eventos, lectura secuencial de bitácoras y análisis estadístico implementado sin bibliotecas especializadas. El sistema utiliza archivos de texto delimitados, escritura en modo *append*, almacenamiento temporal configurable y rotación por tamaño. Se evaluó su rendimiento mediante bitácoras sintéticas de hasta 100 000 eventos. Con un buffer de diez líneas, la escritura tardó 0,2973 s, frente a 0,4560 s con escritura individual. Para 100 000 eventos, la lectura procesó aproximadamente 1,36 millones de eventos por segundo. La memoria máxima rastreada para la lectura perezosa permaneció alrededor de 146 KB, mientras que la carga completa alcanzó 63 930 KB. Los resultados favorecen el procesamiento secuencial, aunque se identificaron limitaciones en algunos módulos estadísticos y en la representatividad de los datos simulados.

I. INTRODUCCIÓN

La telemetría de videojuegos permite evaluar puntajes, duración, daño, progresión y causas de muerte a partir de eventos registrados durante las partidas. Cuando la bitácora crece, cargarla íntegramente en memoria limita la escalabilidad. Este trabajo implementa en Python y Pygame un videojuego *Breakout* con registro de eventos y un motor de análisis de archivos [1]. La visualización utiliza primitivas de Pygame [5]. El objetivo es contrastar escritura con buffer, velocidad de lectura y consumo de memoria frente a la carga completa.

II. DISEÑO DEL SISTEMA

La arquitectura separa `game/`, `telemetry/`, `statistics_engine/`, `tools/` y `tests/`. Se registran los eventos `SESSION_START`, `SESSION_END`, `ENEMY_KILL`, `DAMAGE_TAKEN`, `PLAYER_DEATH` y `LEVEL_UP`. La bitácora usa una línea por evento en el formato `timestamp|session_id|event_type|clave=valor;...`. Se eligió texto para facilitar la inspección y el diagnóstico; su inconveniente es el mayor tamaño frente a formatos binarios. Los delimitadores `|`, `;` y `=` requieren escape o codificación si aparecen dentro de un valor, capacidad todavía no implementada.

El escritor abre archivos con `a`, acumula hasta *N* líneas, aplica `flush()` y escribe el remanente al cerrar. El límite de rotación configurado es 512 KiB en el juego; en las pruebas sintéticas se utiliza un límite mayor. El lector recorre los archivos `events_*.log` ordenados y procesa cada línea mediante un generador. Las líneas vacías, con cantidad incorrecta de campos o con *timestamp* ilegible se descartan; `QualityReport` informa su frecuencia y las sesiones sin cierre. Se decidió **no interpretar una sesión sin `SESSION_END` como partida completa**. Sin embargo, el extractor de sesiones actual todavía incluye sesiones incompletas con valores predeterminados, por lo que necesita un filtro antes de calcular las métricas definitivas. Asimismo, algunos parámetros malformados se omiten parcialmente en lugar de invalidar la línea completa.

III. METODOLOGÍA ESTADÍSTICA

Para cada variable se acumulan conteo, suma, mínimo, máximo, media y un término auxiliar de varianza mediante el método incremental de Welford [2]. Con *n* observaciones, la varianza muestral y el coeficiente de variación son:

$$s^2 = \frac{M_2}{n-1}, \qquad CV = \frac{100 s}{\bar{x}}. \tag{1}$$

El acumulador se actualiza con $\delta = x_n - \bar{x}_{n-1}$, $\bar{x}_n = \bar{x}_{n-1} + \delta/n$ y $M_{2,n} = M_{2,n-1} + \delta(x_n - \bar{x}_n)$. La correlación de Pearson se calcula sin guardar pares:

$$r = \frac{n\sum xy - \sum x \sum y}{\sqrt{(n\sum x^2 - (\sum x)^2)(n\sum y^2 - (\sum y)^2)}}. \tag{2}$$

Se calculan mediana, cuartiles como medianas de las mitades, P90 por rango superior, ocho intervalos de histograma, frecuencias relativas y acumuladas, y atípicos según Tukey [3]:

$$IQR = Q_3 - Q_1, \quad L_{\text{inf}} = Q_1 - 1.5 IQR, \quad L_{\text{sup}} = Q_3 + 1.5 IQR. \tag{3}$$

Para la retención se usa $R_j = N_j / N_1$ y el abandono condicional $A_j = (N_{j-1} - N_j)/N_{j-1}$, donde N_j cuenta sesiones que alcanzaron el nivel *j*.

Tabla I. Complejidad de las estadísticas implementadas

Medida	Una pasada	Memoria auxiliar	Justificación
Media, mínimo y máximo	Sí	O(1)	Acumuladores escalares

Varianza, desviación y CV	Sí	O(1)	Algoritmo de Welford
Correlación de Pearson	Sí	O(1)	Seis sumas y conteo
Frecuencias categóricas	Sí	O(c)	Diccionario de *c* categorías
Histograma, versión actual	No	O(n + k)	Se guardan y ordenan los datos; $k = 8$
Mediana, cuartiles y P90	No	O(n)	Requieren datos ordenados

Como indica la Tabla I, un histograma de límites preestablecidos podría obtenerse con O(k) de memoria; con límites calculados a partir del mínimo y máximo se pueden hacer dos pasadas sin guardar las *n* observaciones. Las medianas aproximadas también podrían estimarse con algoritmos de cuantiles en flujo, a costa de precisión.

IV. METODOLOGÍA EXPERIMENTAL

El generador `synthetic_logs.py` crea sesiones y eventos de combate, daño y progresión con una semilla fija (`seed=42`). El experimento A midió con `time.perf_counter()` la escritura nominal de 100 000 eventos usando buffers de 1, 10, 100 y 1000 líneas. El B midió tiempo de lectura, eventos/s y microsegundos por línea para 5000, 50 000 y 100 000 eventos. El C comparó recorrido perezoso y conversión a lista; `tracemalloc` registró el pico de asignaciones rastreadas por Python [4]. El resumen suministrado identifica **Python 3.14.8, Linux 7.2.9-201.fc44.x86_64 y arquitectura x86_64**; **Intel(R) Core(TM) Ultra 9 285H, 32GB ram**. Los tiempos proceden de una ejecución por configuración; no se reportan repeticiones ni intervalos de confianza.

V. RESULTADOS

Los tiempos medidos en el experimento A se presentan en la Tabla II y la Fig. 1.

Tabla II. Experimento A: tiempo de escritura de 100 000 eventos

Buffer (líneas)	Tiempo (s)
1	0,455950
10	0,297328

100	0,272577
1000	0,266584

Fig. 1. Tiempo de escritura frente al tamaño del buffer para 100 000 eventos. El beneficio marginal disminuye después de diez líneas.

La Tabla III reúne los resultados exigidos de los experimentos B y C; la columna de escritura no se midió por separado.

Tabla III. Resumen consolidado de rendimiento (experimentos B y C)

Eventos	Archivo (KiB)	Escritura (s)	Lectura (s)	Eventos/s	Mem. perezosa (KiB)	Mem. lista (KiB)
5000	261,2	N/D	0,003818	1 309 579	146,2	3307,7
50 000	2613,3	N/D	0,033568	1 489 502	146,3	32 074,9
100 000	5226,3	N/D	0,073791	1 355 183	146,3	63 930,1

Tabla IV. Experimento C: tiempo de recorrido y carga completa

Eventos	Lectura perezosa (s)	Carga en lista (s)
5000	0,005128	0,0079
50 000	0,035051	0,0473
100 000	0,062355	0,0994

En el experimento B se observaron **0,76; 0,67 y 0,74 µs por línea** para 5000, 50 000 y 100 000 eventos, respectivamente.

La Tabla IV evidencia un pico prácticamente constante en la lectura perezosa y crecimiento proporcional al número de eventos al construir la lista: a 100 000 eventos, la segunda operación empleó alrededor de **437 veces** más memoria rastreada. Los resultados del experimento B muestran tiempos de lectura cercanos a la proporcionalidad con el volumen. **No se suministraron tiempos de escritura separados para 5000, 50 000 y 100 000 eventos**; por ello no se completa esa columna de la tabla consolidada exigida en la guía.

VI. DISCUSIÓN

Selección del buffer. Se mantiene $N = 10$, valor usado como predeterminado en el juego. Respecto de $N = 1$, disminuye el tiempo de escritura en **34,8 %**. Aumentarlo de 10 a 100 solo aporta **8,3 %** adicional, a cambio de mantener hasta diez veces más eventos pendientes en memoria antes de vaciarlos. El valor elegido equilibra rendimiento y frecuencia de persistencia; la Fig. 1 sugiere rendimientos decrecientes.

Atípicos. En el conjunto de referencia [1], $Q_1 = 1500$, $Q_3 = 3000$ e $IQR = 1500$. El límite superior de Tukey es 5250, por lo que **9000** es atípico. Su puntaje $z \approx 2,70$ no supera 3: el propio extremo incrementa la media y la desviación estándar, reduciendo su valor *z*. Se prefiere IQR como primera alerta de posibles puntajes anómalos, no como prueba de fraude.

Interpretación de las sesiones disponibles. Se reconstruyeron **30 sesiones completas y 624 eventos** del ejemplo incluido en los archivos adjuntos. Son datos **sintéticos generados por `generate_realistic_logs.py`**, no evidencia de 25 partidas humanas efectivamente jugadas en *Breakout*. El puntaje medio fue **334,47**, frente a mediana **220,5**, señal de asimetría hacia puntajes altos. La correlación duración–puntaje fue **r = 0,95283**, consistente con el generador, donde más acciones ofrecen más oportunidades de puntuar; no demuestra causalidad ni representa necesariamente el comportamiento real de jugadores.

En esa misma muestra, las medias de puntaje fueron **293,67** (*easy*, n = 12), **342,00** (*normal*, n = 8) y **377,40** (*hard*, n = 10), con desviaciones estándar respectivas de **295,33**, **314,04** y **276,38**. La diferencia *hard* − *easy* fue **83,73 puntos (28,5 %)**, insuficiente para afirmar un efecto estable por el tamaño reducido, la dispersión y el origen simulado. Calculando correctamente el embudo por nivel **alcanzado** (no por nivel final exacto), el mayor abandono condicional se observó entre los niveles 9 y 10: **5 de 8 sesiones, 62,5 %**. Como cambio a **validar con partidas reales**, se propone reducir temporalmente la densidad de ladrillos del nivel 10 y volver a medir su retención.

Limitaciones de implementación. La lectura de eventos individuales es perezosa, pero `iter_sessions()` conserva un diccionario por sesión y la pantalla de estadísticas materializa listas; por tanto, **no todo el análisis tiene memoria O(1)**. Además, `level_funnel()` cuenta niveles finales exactos, por lo que debe corregirse para contar niveles alcanzados. Se requieren pruebas con bitácoras reales, mediciones repetidas y validación del cierre de sesiones antes de afirmar el cumplimiento integral del laboratorio.

VII. CONCLUSIONES

El prototipo demuestra escritura incremental, análisis estadístico manual y lectura secuencial de bitácoras con bajo consumo de memoria para el caso medido. Un buffer de diez líneas redujo sustancialmente el costo de escritura frente al envío individual; la lectura perezosa evitó el crecimiento observado al cargar listas completas. Queda pendiente corregir la agregación por sesión y el embudo, reforzar el tratamiento de registros corruptos, documentar el hardware, realizar pruebas repetidas y obtener al menos 25 sesiones jugadas realmente. Como trabajo futuro se contempla una mediana aproximada en flujo y una evaluación de retención con usuarios.

REFERENCIAS

[1] D. J. Romero Martínez, *Laboratorio: Estructura de Datos I*, Departamento de Ingeniería de Sistemas, Universidad del Norte, guía de laboratorio suministrada en clase, s. f.

[2] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962, doi: 10.1080/00401706.1962.10490022.

[3] J. W. Tukey, *Exploratory Data Analysis*. Reading, MA, USA: Addison-Wesley, 1977.

[4] Python Software Foundation, "`tracemalloc` — Trace memory allocations," *Python Documentation*. [En línea]. Disponible: https://docs.python.org/3/library/tracemalloc.html

[5] Pygame Community, "`pygame.draw` — Pygame documentation." [En línea]. Disponible: https://www.pygame.org/docs/ref/draw.html